UPF

INSTITUTO DE TECNOLOGIA - ITec

Algoritmos e Programação

Prof. Eder Pazinatto

Estruturas de Repetição - Listas

Estruturas de Dados - LISTAS

- As **Listas** são estruturas de dados utilizadas para armazenar diversos valores em uma única variável.

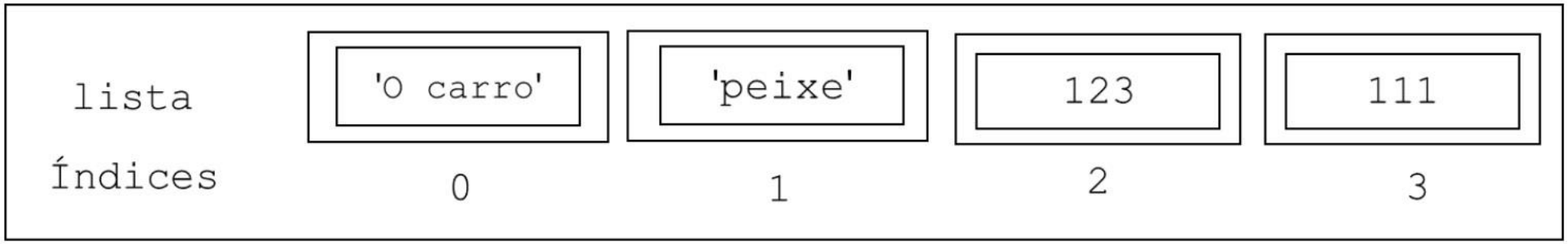
- Seus elementos são organizados sequencialmente e acessados por meio do índice correspondente ao item ou por meio de métodos para a manipulação de listas disponíveis na linguagem

- Os elementos de uma lista são delimitados pelos sinais de colchetes: [ ].

Estruturas de Dados - LISTAS

- É possível armazenar diversos tipos de dados em uma lista, como *strings*, *inteiros*, *bool* e, inclusive, *outras listas*.

```
lista = ['O carro','peixe',123,111]
```



- O primeiro elemento da lista (*O Carro*) está na posição ZERO (0).

Estruturas de Dados - LISTAS

- Para CRIAR listas no Python é necessário:
  - ✓ Utilizar o símbolo [ ] (colchetes) para as listas;
  - ✓ Armazenar a lista em uma VARIÁVEL;

```
#Criando uma lista com elementos...
listaCompras = ['arroz','feijão','massa']
print(listaCompras)
```

```
>>> %Run Listas.py
 ['arroz', 'feijão', 'massa']
```

Estruturas de Dados - LISTAS

- Criar uma lista contendo elementos e outra listas.

```
listaCompras = ['arroz','feijão','massa']
minhaLista = [1.2, 555, True, listaCompras]
print(minhaLista)
```

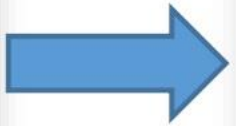


```
>>> %Run Listas.py
 [1.2, 555, True, ['arroz', 'feijão', 'massa']]
```

Estruturas de Dados - LISTAS

- Acessando um elemento da lista, considerando o índice da lista.

```
1  lista = [9, 4, 5]
2  #acessando o item 0 da lista
3  print (lista[0])
4  #acessando o item 1 da lista
5  print (lista[1])
6  #acessando o item 2 da lista
7  print (lista[2])
8  # apresenta o intervalo de 0 a 1
9  print (lista[0:2])
10 # apresenta o intervalo de 0 a 1
11 print (lista[:2])
12 # apresenta o intervalo de 1 a 2
13 print (lista[1:])
```



```
Shell
>>> %Run Lista_1.py

   9
   4
   5
   [9, 4]
   [9, 4]
   [4, 5]
```

Estruturas de Dados – LISTAS : operadores

- **Concatenando Listas**: junta os elementos de duas ou mais listas em uma saída.

```
#Concatenando listas
livros = ['Java','SqlServer', 'Delphi','Python']
revistas = ['Java Magazine','Mundo Delphi','Explorando Python']
listaConcatenada = livros + revistas
print(listaConcatenada)
```



```
>>> %Run Listas.py
 ['Java', 'SqlServer', 'Delphi', 'Python', 'Java Magazine', 'Mundo Delphi', 'Explorando Python']
```

Estruturas de Dados – LISTAS : métodos/funções

- Verificando a existência de elementos na lista: retorna **True** ou **False** caso encontre ou não o elemento procurado na lista.

```
#Verificando a existência de um elemento na lista
livros = ['Java','SqlServer', 'Delphi','Python', 'Java']
print( 'Java' in livros)
```

Saída: **True**

- Utilizando funções para valores mínimo, máximo e soma na lista.

```
#buscando valores mínimo, máximo e a soma dos elementos na lista
listaNumeros = [10.50, 20, 10, 30.50]
print( min(listaNumeros) )
print( max(listaNumeros) )
print( sum(listaNumeros) )
```

Saída: **10, 30.5, 71**

Estruturas de Dados – LISTAS : métodos/funções

- **len()** – Retorna o comprimento/tamanho da lista.

```
#Retornando o comprimento da lista
livros = ['Java','SqlServer', 'Delphi','Python']
print(len(livros))
```

- **count()** – Retorna o número de ocorrências de um determinado objeto que foi passado como parâmetro.

```
#Retornando o numero de ocorrências do elemento Java na lista
livros = ['Java','SqlServer', 'Delphi','Python', 'Java']
print(livros)
print(livros.count('Java'))
```

- **sort()** – Ordena uma lista (ordem crescente alfabética).

livros.sort() # ordem decrescente livros.sort(reverse = True)

Estruturas de Dados – LISTAS : métodos/funções

- **append()** – Recebe 3 valores via teclado e armazena na lista.

```
for i in range(3):
    valor = int(input(f"Digite o {i + 1}º número: "))
    numeros.append(valor)
```

- **Insert()** – insere o valor 20 na posição 1 da lista

```
numeros.insert(1, 20)
```

- Verifica se todos valores **são positivos**, retorna booleano

```
print("Todos são positivos?", all(n > 0 for n in numeros))
```

- Verifica se há valores na lista **maior que 30**, retorna booleano

```
print("Existe valor maior que 30?", any(n > 30 for n in numeros))
```

Estruturas de Dados – LISTAS: métodos/funções

- **pop(*i*) -** Remove o último item de lista. Se a posição [i] for informada, remove o item correspondente.

  livros.pop()

  livros.pop(2) # remove da posição 2

- **clear() -** Remove todos os itens de uma lista

  livros.clear()

- **reverse() –** Inverte as posições dos itens.

  livros.reverse() # esta na posição 4, passa para a zero(0), esta na 3 vai a pos. (1)